



IES ALBARREGAS

Departamento de Informática

**TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES
MULTIPLATAFORMA**



MEMORIA DE PROYECTO

Autor: Juan Eloy Ortiz Lara.

Nombre del proyecto: PadelPro creado por Club007.

Grupo: DAM 2B.

Tutor/a: María Mercedes Martínez Fragoso.

Curso académico 2025/2026

Fecha y versión: v1.0 a 4/11/2025

Enlaces importantes: TRELLO, FIGMA y GITHUB

Repositorio: <https://github.com/juane77/PadelPro-TFG>

Licencias: GPL GNU v3.

Queda por escrito que este trabajo es de carácter educativo por lo que no podrá ser utilizado para ningún otro fin sin el consentimiento del autor y del centro educativo.

Índice

1. Introducción
2. Objetivos del proyecto
3. Metodología de gestión del proyecto
4. Análisis y planificación
 - 4.1. Requisitos funcionales
 - 4.2. Requisitos no funcionales
 - 4.3. Planificación temporal
5. Arquitectura del sistema
 - 5.1. Frontend — Flutter
 - 5.2. Backend — Spring Boot
 - 5.3. Base de datos — PostgreSQL / Supabase
 - 5.4. Servicios externos
 - 5.5. Despliegue e infraestructura
6. Proceso de desarrollo
 - 6.1. Iteraciones y fases
 - 6.2. Dificultades encontradas y soluciones
7. Resultados obtenidos
8. Propuesta de mejoras futuras
9. Conclusiones
10. Bibliografía

1. Introducción

El pádel lleva años haciéndose fuerte y conocido en España y yo, como aficionado al deporte, me di cuenta de que la mayoría de clubs y jugadores siguen gestionando sus reservas por WhatsApp o incluso en papel. Cuando llegó el momento de elegir el proyecto del TFG, vi ahí una oportunidad clara.

PadelPro es una aplicación multiplataforma que permite a los jugadores reservar pistas, registrar sus partidos y conectar con otros usuarios, mientras que los clubs disponen de un panel de administración para gestionar toda su actividad. Funciona tanto en Android como en web desde cualquier navegador.

Para desarrollarla he aplicado los conocimientos adquiridos a lo largo del ciclo: Flutter y Dart para el frontend, Java con Spring Boot para el backend, PostgreSQL como base de datos y varios servicios en la nube para el despliegue. Ha sido un reto importante pero también la experiencia más completa que he tenido como desarrollador hasta ahora, ya que he tenido que tomar decisiones reales de arquitectura, resolver problemas de integración y poner en producción una aplicación funcional desde cero.

2. Objetivos del proyecto

Desde el principio tuve claro lo que quería conseguir con este proyecto. No quería hacer algo que solo funcionara en local o que fuera un ejercicio de clase más, quería que fuera una aplicación real, que cualquier persona pudiera descargar, instalar y usar sin que le diera problemas.

A partir de ahí me fui marcando objetivos más concretos según avanzaba el desarrollo:

- Lo primero era tener una app Android funcional hecha con Flutter que cubriera todo lo que un jugador de pádel puede necesitar en el día a día, desde reservar una pista hasta ver cómo está rindiendo en sus partidos.
- También quería que no hiciera falta tener un móvil Android para probarla, así que me propuse publicar una versión web accesible desde cualquier navegador.
- Por detrás necesitaba un backend que aguantara todo eso, así que desarrollé una API REST con Spring Boot que centraliza toda la lógica de negocio y es la que consume el frontend.
- Para los datos usé PostgreSQL alojado en Supabase, que me permitió tener una base de datos gestionada en la nube sin complicarme con configuraciones de servidor.
- Quería también integrar servicios externos de verdad, no simulados: Brevo para los emails de verificación y recuperación de contraseña, y Supabase Storage para las fotos de perfil.
- Todo desplegado en la nube desde el principio: el backend en Render y la web en Vercel, con despliegue automático desde GitHub.
- Un sistema de roles que diferenciara entre usuarios normales, administradores de club y superadministrador, con permisos distintos para cada uno.
- Y por último, aunque no menos importante, que la app tuviera un diseño cuidado y coherente, con una identidad visual propia que le diera personalidad a PadelPro.

3. Metodología de gestión del proyecto

El proyecto se ha desarrollado siguiendo una metodología ágil basada en iteraciones cortas, similar a Scrum, aunque adaptada al contexto de un proyecto individual de TFG. Cada iteración tuvo una duración aproximada de una a dos semanas y se orientó a entregar un incremento funcional del producto.

Para la gestión de tareas e historias de usuario se utilizó Trello como herramienta de tablero Kanban, organizando las tareas en columnas de Pendiente, En progreso y Completado. Las historias de usuario se definieron desde el análisis inicial y sirvieron como guía para priorizar las funcionalidades a implementar en cada iteración.

El control de versiones del código se gestionó íntegramente mediante Git y GitHub, realizando commits frecuentes con mensajes descriptivos que permiten seguir la evolución del proyecto.

Se adoptaron las siguientes prácticas durante el desarrollo:

- Separación clara entre frontend (Flutter) y backend (Spring Boot) mediante una API REST.
- Uso de variables de entorno para las credenciales sensibles, evitando datos hardcodeados en el código.
- Despliegue continuo: cada push a la rama principal actualiza automáticamente el backend en Render y la web en Vercel.
- Pruebas manuales exhaustivas en dispositivo físico Android y navegador web Chrome.

4. Análisis y planificación

4.1. Requisitos funcionales

Los requisitos funcionales principales identificados durante la fase de análisis son:

- RF01 — Registro e inicio de sesión con verificación de email mediante código de 6 dígitos.
- RF02 — Recuperación de contraseña mediante código enviado por email.
- RF03 — Reserva de pistas con selección de fecha y hora disponible.
- RF04 — Visualización y cancelación de reservas activas.
- RF05 — Registro de partidos con marcador, nivel, resultado y amigos participantes.
- RF06 — Estadísticas de rendimiento: porcentaje de victorias, racha actual, mejor racha y nivel medio.
- RF07 — Sistema de amistades con solicitudes, aceptación y rechazo.
- RF08 — Feed de noticias de pádel en tiempo real mediante integración con API externa.

- RF09 — Mapa interactivo con localización de clubs cercanos.
- RF10 — Perfil de usuario con foto, nombre y saldo de pelotas virtuales.
- RF11 — Sistema de pelotas como moneda virtual: +5 por inicio de sesión diario, -15 por reserva, +10 por cancelación.
- RF12 — Panel de administración web para superadmin y admins de club con gestión de usuarios y reservas.
- RF13 — Subida de foto de perfil a Supabase Storage visible para otros usuarios.
- RF14 — Notificaciones push tras la reserva o cancelación de alguna reserva, al igual que por cualquier problema.

4.2. Requisitos no funcionales

- RNF01 — La aplicación debe funcionar en Android (APK) y en web (navegadores modernos).
- RNF02 — El tiempo de respuesta del backend no debe superar los 3 segundos en condiciones normales.
- RNF03 — Las contraseñas deben almacenarse cifradas con BCrypt.
- RNF04 — La autenticación se realizará mediante tokens JWT con expiración de 24 horas.
- RNF05 — El sistema debe soportar múltiples usuarios simultáneos sin degradación del rendimiento.
- RNF06 — El diseño debe ser responsive y adaptarse a diferentes tamaños de pantalla.

4.3. Planificación temporal

El desarrollo del proyecto se organizó en las siguientes fases:

Fase	Descripción	Duración
Fase 1	Análisis de requisitos y diseño de la base de datos	2 semanas
Fase 2	Desarrollo del backend: modelos, repositorios, controladores y JWT	4 semanas
Fase 3	Desarrollo del frontend Flutter: pantallas principales	5 semanas
Fase 4	Integración frontend-backend y pruebas	3 semanas

Fase 5	Despliegue en Render, Vercel y configuración de Supabase	1 semana
Fase 6	Corrección de bugs, documentación y preparación de la presentación	2 semanas

5. Arquitectura del sistema

PadelPro sigue una arquitectura cliente-servidor desacoplada. El frontend (Flutter) se comunica con el backend (Spring Boot) exclusivamente a través de una API REST con autenticación JWT. A continuación se describen brevemente los componentes del sistema.

5.1. Frontend — Flutter

El frontend está desarrollado con Flutter 3.38.5 y Dart 3.10.4, lo que permite generar tanto la APK de Android como la versión web desde la misma base de código. La arquitectura de pantallas sigue el patrón de StatefulWidget con gestión de estado mediante Provider. La navegación entre pantallas se realiza con MaterialPageRoute.

Los servicios de acceso a datos (ApiService, ReservaService, PartidoService, AmistadService, etc.) encapsulan las llamadas HTTP al backend. La sesión del usuario se gestiona mediante la clase estática Session, que almacena en memoria el token JWT, el id de usuario, nombre, email, rol, saldo de pelotas y foto de perfil.

5.2. Backend — Spring Boot

El backend está desarrollado con Java 21 y Spring Boot 4.0.2, siguiendo la arquitectura MVC con las capas de Controlador, Servicio, Repositorio y Modelo. La seguridad está implementada con Spring Security y autenticación JWT mediante filtros personalizados.

Los principales módulos del backend son: gestión de usuarios (registro, login, verificación de email, recuperación de contraseña), gestión de reservas (creación, cancelación, scheduler automático), gestión de partidos, gestión de amistades y notificaciones, y panel de administración con endpoints protegidos por rol.

5.3. Base de datos — PostgreSQL / Supabase

La base de datos es PostgreSQL, alojada en Supabase. El acceso se realiza mediante Hibernate/JPA con Spring Data JPA. Las principales entidades del modelo de datos son: Usuario, Pista, Club, Reserva, Partido, Amistad, Notificacion y SolicitudAmistad.

5.4. Servicios externos

Brevo	Envío de emails transaccionales (verificación de cuenta y recuperación de contraseña) mediante API HTTP. Se eligió frente a SMTP por las restricciones del plan gratuito de Render.
Supabase Storage	Almacenamiento de fotos de perfil de usuarios. Las imágenes se suben mediante peticiones HTTP a la API de Supabase y se sirven como URLs públicas.
NewsAPI	Integración para mostrar noticias actualizadas del mundo del pádel en la sección de Noticias de la app.
OpenStreetMap flutter_map	/ Mapa interactivo para mostrar la localización de clubs de pádel cercanos al usuario.

5.5. Despliegue e infraestructura

Backend	Render (padelpro-tfg.onrender.com) — despliegue automático desde GitHub con Docker.
Web Flutter	Vercel (padel-pro-tfg.vercel.app) — despliegue automático desde GitHub.
Landing page	Cloudflare Workers (twilight-sunset-79c3.juaneloyortizlara.workers.dev)
Base de datos	Supabase (vketxcsgjfpbgrgxwopv.supabase.co) — PostgreSQL gestionado.
Almacenamiento	Supabase Storage — bucket 'avatares' público.
APK Android	Google Drive — enlace de descarga directa desde la landing page.

6. Proceso de desarrollo

6.1. Iteraciones y fases

El desarrollo comenzó con la configuración del proyecto Spring Boot, la definición del modelo de base de datos y la implementación de los endpoints básicos de autenticación. A continuación se desarrolló el frontend Flutter comenzando por las pantallas de login y registro, y progresivamente se fueron añadiendo las funcionalidades de reservas, partidos, amistades y noticias.

En paralelo al desarrollo funcional se fue configurando la infraestructura de despliegue, lo que permitió probar la integración real entre frontend y backend desde etapas tempranas. El panel de administración web se desarrolló como un archivo HTML estático servido por el propio backend de Spring Boot.

6.2. Dificultades encontradas y soluciones

- SMTP bloqueado por Render: El proveedor de hosting bloquea las conexiones SMTP salientes. Solución: migrar el envío de emails a la API HTTP de Brevo.
- Imagen gris en Flutter Web: AssetImage dentro de DecorationImage no renderiza en web. Solución: diagnosticar con console log, identificar el error real (acceso anidado a datos del backend) y corregir el parseo de las reservas.
- Datos anidados del backend: El frontend intentaba acceder a `reserva['pista']['nombre']` pero el backend devolvía un mapa plano. Solución: unificar el formato de respuesta en el DTO del controlador de reservas.
- Foto de perfil entre pantallas: La foto se perdía al navegar. Solución: añadir `Session.fotoPerfil` como campo estático compartido entre todas las pantallas.
- Contexto agotado en despliegue de Netlify: La landing page se desplegaba en Netlify con créditos limitados. Solución: migrar a Cloudflare Workers que ofrece despliegue gratuito ilimitado.

7. Resultados obtenidos

Al finalizar el desarrollo, PadelPro ofrece las siguientes funcionalidades completamente operativas:

- Sistema completo de registro, login y verificación de email con código.
- Reserva y cancelación de pistas con sistema de pelotas virtuales.
- Registro de partidos con estadísticas de rendimiento.
- Sistema de amistades con solicitudes y notificaciones.
- Feed de noticias de pádel en tiempo real.
- Mapa interactivo de clubs cercanos.
- Subida y visualización de foto de perfil a través de Supabase Storage.
- Panel de administración web con gestión de usuarios y reservas por rol.
- Versión APK para Android y versión web accesible desde cualquier navegador.
- Toda la infraestructura desplegada en la nube con despliegue continuo.

8. Propuesta de mejoras futuras

- Chat en tiempo real entre jugadores utilizando WebSockets o Firebase Realtime Database.
- Sistema de torneos: organización de partidos por eliminatorias con cuadros de resultados.
- Ranking de jugadores por nivel, victorias o pelotas acumuladas.
- Integración con pasarela de pago para reservas con pago real.
- Soporte para iOS mediante compilación nativa con Flutter.
- Sistema de valoraciones de clubs por parte de los usuarios.
- Modo oscuro en la aplicación.

9. Conclusiones

El desarrollo de PadelPro ha supuesto un reto técnico completo que ha permitido aplicar y profundizar en los conocimientos adquiridos a lo largo del ciclo formativo de 2DAM. El proyecto integra tecnologías modernas de desarrollo multiplataforma (Flutter), desarrollo backend (Spring Boot), gestión de bases de datos (PostgreSQL/Supabase) y despliegue en la nube (Render, Vercel, Cloudflare), conformando un stack tecnológico actual y ampliamente utilizado en la industria.

El resultado es una aplicación funcional, desplegada y accesible al público, lo que representa el objetivo final de cualquier proyecto de desarrollo software real. Las dificultades encontradas durante el desarrollo han sido oportunidades de aprendizaje que han enriquecido el proceso y han demostrado la capacidad de resolución de problemas en entornos de producción reales.

En definitiva, PadelPro es un proyecto del que me siento orgulloso tanto por su funcionalidad como por la solidez técnica de su implementación, y que sienta las bases para futuras mejoras y ampliaciones.

10. Bibliografía

- Documentación oficial de Flutter: <https://flutter.dev/docs>
- Documentación oficial de Spring Boot: <https://spring.io/projects/spring-boot>
- Documentación de Supabase: <https://supabase.com/docs>
- Documentación de Brevo API: <https://developers.brevo.com>
- StackOverflow: <https://stackoverflow.com>
- Render Docs: <https://render.com/docs>
- Vercel Docs: <https://vercel.com/docs>
- pub.dev (paquetes Flutter): <https://pub.dev>
- GitHub del proyecto: <https://github.com/juane77/PadelPro-TFG>